X728 UPS Auto Shutdown Installation Guide

What This Guide Will Help You Do

This guide sets up your Raspberry Pi so it shuts down safely when the power goes out, and then powers off the X728 UPS too, so the battery is not drained.

What you'll need:

- Raspberry Pi 4B with the X728 UPS board attached
- Keyboard, mouse and monitor connected to your Pi
- Internet connection
- About 30 minutes

What it does:

- ✅ **Safe shutdown:** 5 seconds after power is lost, the Pi shuts down cleanly.
- ✅ **Short dips are ignored:** if power comes back during the 5-second countdown (for example while a car engine starts), the shutdown is cancelled.
- ✅ **Complete power down:** after the Pi shuts down, the UPS turns itself off to save the battery.
- ✅ **Normal reboots stay normal:** `sudo reboot` with power connected does not turn off the UPS.
- ✅ **Auto power-on:** when power returns, the Pi starts again (needs the AUTO ON jumper).

📋 Step 1: Check Your Hardware Setup

1.1 Physical Connections

1. Turn off your Pi and unplug the power.
2. Check the X728 board is properly connected to your Pi's GPIO pins.
3. Check the jumpers on the X728:
    - The **AUTO ON** jumper is connected (allows auto power-on).
    - The **CHG EN** jumper is connected (enables charging).
4. Connect your power supply to the X728 board (**not** directly to the Pi).
5. Insert the 18650 batteries into the X728 if you haven't already.

1.2 Power On Test

1. Plug in the power to the X728 board.
2. Press the power button on the X728 to turn on your Pi.
3. Wait for the Pi to boot completely.

✅ **Success check:** your Pi boots normally and you see the desktop.

🛠️ Step 2: Enable the I2C Interface

The X728 board talks to your Pi using “I2C”. We need to turn it on.

2.1 Open the Configuration Tool

1. Open a terminal (the black square icon on the desktop).
2. Run:

   ```
   sudo raspi-config
   ```

3. Enter your password if asked.

2.2 Enable I2C

1. Use the arrow keys to go to **Interface Options** and press Enter.
2. Go to **I2C** and press Enter.
3. Select **Yes** when asked “Would you like the ARM I2C interface to be enabled?”
4. Select **OK**, then **Finish**.
5. Select **Yes** when asked to reboot.

2.3 Verify I2C Works

After the reboot, open a terminal again and run:

```
sudo i2cdetect -y 1
```

✅ **Success check:** you see **36** somewhere in the grid. That is your X728 board.

💾 Step 3: Install the Required Software

3.1 Update Your System

```
sudo apt update
sudo apt upgrade -y
```

This might take a few minutes.

3.2 Install the Required Libraries

```
sudo apt install -y git python3-smbus python3-rpi.gpio gpiod
```

What these do:

- `git`: downloads the project files.
- `python3-smbus`: lets Python talk to the X728 over I2C.
- `python3-rpi.gpio`: lets the script read the power-loss pin.
- `gpiod`: sends the "power off" signal to the UPS (same method as the official Geekworm scripts).

✅ **Success check:** the installation finishes without errors.

📁 Step 4: Get the Script

4.1 Download the Project

```
cd ~
git clone https://github.com/brandespinosa/RiAct-Carplay.git
```

4.2 Copy the Files Into Place

```
mkdir -p ~/x728_power_monitor
cp ~/RiAct-Carplay/x728_auto_shutdown.py ~/RiAct-Carplay/x728-power-monitor.service
~/x728_power_monitor/
cd ~/x728_power_monitor
chmod +x x728_auto_shutdown.py
```

🧪 Step 5: Test the Script

5.1 Run Test Mode

```
python3 x728_auto_shutdown.py
```

What you should see:

- “X728 UPS Power Monitor Starting…”
- “AC Power: OK” if your power is connected
- “Press Ctrl+C to stop monitoring”

> If you start the script while the power is **unplugged**, it starts the 5-second shutdown countdown right away. That is on purpose: it stops the Pi from running on battery until the battery is empty.

5.2 Test Power Loss Detection

1. With the script running, carefully unplug the power adapter from the X728.
2. You should see:
   - “⚠️ AC POWER LOSS DETECTED!”
   - A countdown: “Shutting down system in 5 seconds…”
3. Plug the power back in before the countdown ends. You should see “✅ AC POWER RESTORED!”, and the shutdown is cancelled.
4. To test a full shutdown, unplug the power and let the countdown finish. The Pi shuts down safely.

> **Note:** in this manual test **only the Pi** shuts down. The UPS stays on, because the systemd service you set up in Step 6 powers off the UPS. Press the X728 power button to turn the Pi back on.

5.3 Stop Test Mode

Press **Ctrl + C** to stop the script.

✅ **Success check:** the script detects power loss, starts the countdown, and cancels it when power comes back.

🚀 Step 6: Set Up Automatic Start

6.1 Set Your Username

The service file uses the username `pi`. Raspberry Pi OS Bookworm has no `pi` user by default, so replace it with your own username:

```
cd ~/x728_power_monitor
sed -i "s/\bpi\b/$USER/g" x728-power-monitor.service
```

Check the result:

```
cat x728-power-monitor.service
```

The `User=` line and the `/home/.../x728_auto_shutdown.py` path should now show your username.

6.2 What the Service File Does

```
[Unit]
Description=X728 UPS Power Monitor
After=multi-user.target
StartLimitIntervalSec=0

[Service]
Type=simple
Restart=always
RestartSec=1
# Change "pi" to your username (Bookworm has no default pi user)
User=pi
ExecStart=/usr/bin/python3 /home/pi/x728_power_monitor/x728_auto_shutdown.py
# Power off the UPS only when AC is really lost (GPIO6 HIGH), not on reboot/restart
ExecStop=/bin/sh -c '[ "$$(gpioget 0 6)" = 1 ] && gpioset 0 26=1 || true'
ExecStopPost=/bin/sleep 4
TimeoutStopSec=10
KillMode=mixed
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

Key lines:

- `ExecStop=...`: when the service stops during shutdown, this sends the “power off” signal to the UPS (`gpioset 0 26=1`). It does this **only if the power is really lost**

(GPIO 6 reads 1). A normal `sudo reboot` or a service restart with power connected does **not** turn off the UPS.
- `ExecStopPost=/bin/sleep 4`: gives the UPS time to register the signal.
- `TimeoutStopSec=10`: stops systemd from killing the stop step too quickly.
- `Restart=always`: restarts the script if it ever crashes.

6.3 Install and Enable the Service

```
sudo cp x728-power-monitor.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable x728-power-monitor.service
sudo systemctl start x728-power-monitor.service
```

6.4 Check the Service Status

```
sudo systemctl status x728-power-monitor.service
```

✅ **Success check:** the status shows **active (running)** in green.

📊 Step 7: Monitor and Logs

7.1 View Live Logs

```
sudo journalctl -u x728-power-monitor.service -f
```

Press **Ctrl + C** to stop viewing.

7.2 View the Log File

```
tail -f ~/x728_power_monitor/x728_shutdown.log
```

7.3 Log Management

✅ **Automatic log rotation is built in:**

- When the log file reaches about 100KB, it is renamed to `x728_shutdown.log.1` and a new log file starts.
- Only one old file is kept, so the logs never use more than about 200KB.
- No manual maintenance is needed.

7.4 Check if the Service Is Running

```
sudo systemctl is-active x728-power-monitor.service
```

🧪 Step 8: Final Testing

8.1 Test Automatic Startup

1. Reboot your Pi with the power connected:

   ```
   sudo reboot
   ```

2. The Pi should restart normally. The UPS stays on.

3. After the reboot, check that the service started:

   ```
   sudo systemctl status x728-power-monitor.service
   ```

8.2 Test Power Loss Protection

1. Check the logs to confirm monitoring:

   ```
   sudo journalctl -u x728-power-monitor.service --since "1 minute ago"
   ```

2. Unplug the power for about 2 seconds, then plug it back in. The shutdown should be cancelled (“AC POWER RESTORED” in the log).

3. Unplug the power and leave it unplugged. After 5 seconds the Pi shuts down, then the UPS powers off (all LEDs off).

4. Plug the power back in. The Pi should start automatically.

🔧 Troubleshooting

Problem: “Permission denied” errors

Make sure you use sudo for system commands:

```
sudo systemctl status x728-power-monitor.service
```

Problem: GPIO permission errors in the log

Add your user to the `gpio` group, then reboot:

```
sudo usermod -a -G gpio $USER
sudo reboot
```

Problem: Script can't find GPIO pins

Check I2C is enabled and the X728 is properly connected:

```
sudo i2cdetect -y 1
```

Problem: Service won't start

- Check the status and logs:

  ```
  sudo systemctl status x728-power-monitor.service
  sudo journalctl -u x728-power-monitor.service
  ```

- Make sure the `User=` line and the script path in the service file match your username (Step 6.1). Show the installed file with:

  ```
  systemctl cat x728-power-monitor.service
  ```

Problem: The Pi shuts down, but the UPS stays on

1. Make sure `gpiod` is installed:

   ```
   sudo apt install -y gpiod
   ```

2. Check the service has the power-off step:

   ```
   sudo systemctl show x728-power-monitor.service -p ExecStop
   ```

   It should mention `gpioset 0 26=1`.

3. Test the power-off signal by hand. **Warning: this turns the whole system off.** Save your work first.

   ```
   gpioset 0 26=1
   ```

Problem: False power loss detections

Increase the debounce time near the top of `x728_auto_shutdown.py`:

```
DEBOUNCE_TIME = 1.0  # Increase from 0.5 to 1.0 seconds
```

Then restart the service:

```
sudo systemctl restart x728-power-monitor.service
```

📱 Useful Commands Reference

Service Management

```
# Start the service
sudo systemctl start x728-power-monitor.service

# Stop the service
sudo systemctl stop x728-power-monitor.service

# Restart the service
sudo systemctl restart x728-power-monitor.service

# Check if the service is running
sudo systemctl is-active x728-power-monitor.service

# View the service status
sudo systemctl status x728-power-monitor.service

# Disable auto-start
sudo systemctl disable x728-power-monitor.service

# Enable auto-start
sudo systemctl enable x728-power-monitor.service
```

Log Viewing

```
# View recent logs
sudo journalctl -u x728-power-monitor.service --since "10 minutes ago"

# View live logs
sudo journalctl -u x728-power-monitor.service -f

# View the log file
tail -f ~/x728_power_monitor/x728_shutdown.log

# Clear the log manually (rarely needed)
> ~/x728_power_monitor/x728_shutdown.log

# Check the log file size
ls -lh ~/x728_power_monitor/
```

🎉 Congratulations!

Your Raspberry Pi and X728 UPS are now protected against power loss.

What happens now:

- **Normal operation:** your Pi runs normally with UPS protection.
- **Short power dip:** ignored if the power comes back within 5 seconds.
- **Power loss:** after 5 seconds the Pi shuts down safely, then the UPS powers off to save the battery.
- **Started without power:** the Pi shuts down after 5 seconds instead of draining the battery.
- **Reboot:** a normal reboot with power connected does not turn off the UPS.
- **Power restored:** the Pi turns back on automatically (if the AUTO ON jumper is connected).
- **Logging:** all events are logged, and the log manages its own size.